

HW11: SSA Quad 到 ARM Assembly 指令转化

- 姓名：陈雨萱
- 学号：23300240034
- 专业：计算机科学与技术（拔尖人才计划）

关键技术实现

这一次的实验主要是把 SSA 形式的中间代码Quad翻译成 ARMv7-A 汇编，输出 .s 文件，完成编译器后端的最后一步——代码生成。输入是 `.4-ssa-withflow-xml.quad`，输出是接近可运行的汇编。我实现了流水线里的三个模块。`buildAdvDFG.cc` 在每个基本块内建依赖图，表示哪些 Quad 必须先执行。`selectInstr.cc` 把 MOVE、LOAD/STORE、CALL、PTR_CALC 等 Quad 映射成 ARM 指令，并在依赖约束下做列表调度，决定块内指令顺序。`schedule.cc` 把各基本块拼成完整函数，处理序言/尾声、PHI（边上用 mov）、跳转、return 和 exit。这个实验把前面学过的 SSA、基本块、数据流，落到真实的指令选择与调度上；同时涉及调用约定、寻址优化、虚调用等后端细节。13 个测试全部通过，说明你的编译器能从 Quad 稳定生成与参考一致的 ARM 汇编。具体三个文件的实现思路如下。

buildAdvDFG.cc

该代码文件主要是从已经生成好的Quad中间表示出发，为每个基本块构建高级数据流图AdvDFG。构建过程中不仅保留了语句之间的顺序依赖关系，还额外建立了变量定义-使用依赖和内存地址计算链依赖，从而为后续优化分析提供更精确的数据流信息。各个函数的作用如下：

1. `isExitCallStmt`函数用于判断一条语句是否本质上表示程序退出，即 `exit` 调用。由于 Quad IR 中函数调用存在多种表示形式如 `CALL`、`MOVE_CALL`、`EXTCALL`、`MOVE_EXTCALL`，因此函数需要分别检查不同类型的调用节点，并判断其目标函数名是否为 `"exit"`。这样做的目的是在构建数据流图时，将这些语句视为控制流终止点，与普通函数调用进行区分。
2. `isBlockFinalStmt`函数用于判断一条语句是否是基本块的结束语句。判断条件包括：
 - `RETURN`
 - `JUMP`
 - `CJUMP`
 - `exit` 调用

如果满足上述条件，则说明该语句会终止当前基本块的正常执行流程。在构建数据流图时，这类语句会被标记为 `ExitStatement` 类型节点，以体现其特殊的控制流语义。

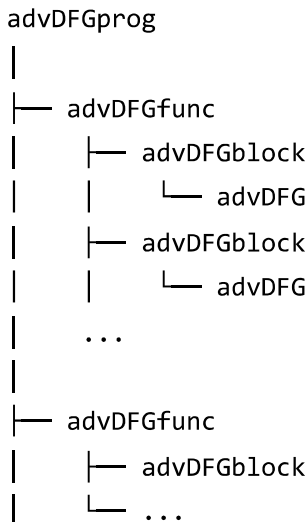
3. `singleDefTempNum`函数用于提取一条语句所定义的临时变量编号。由于 Quad IR 中每条语句都维护了自己的定义集合，函数首先检查该集合是否为空，然后返回其中唯一变量的编号。如果语

句没有定义变量，则返回 -1。该函数的作用是后续建立 Def-Use 数据依赖关系提供定义点信息。

4. useTempNums函数用于收集一条语句所使用的所有临时变量编号。函数遍历语句的 use 集合，并将所有使用到的变量编号保存到一个 set<int> 中返回。这些变量编号将在后续构建数据流图时用于寻找对应的定义节点，从而建立 Def-Use 依赖边。
5. isPtrCalcStmt函数用于判断当前语句是否属于指针地址计算（PTR_CALC）。在本实验实现中，地址计算被单独视为一种特殊的数据流链，因为后续的 LOAD 和 STORE 操作依赖于这些地址计算结果。因此需要首先识别出这类语句。
6. isLoadOrStoreStmt该函数用于判断当前语句是否为内存访问操作。具体包括 LOAD 和 STORE。由于内存访问不仅依赖于数据本身，还依赖于访问地址，因此后续需要利用该函数筛选出需要建立地址依赖关系的节点。
7. ptrCalcMemTempNum函数用于从 LOAD 或 STORE 语句中提取用于表示内存地址的临时变量编号。具体而言：对于 LOAD，读取其源地址变量；对于 STORE，读取其目标地址变量；如果地址不是 Temp 类型，则返回 -1。该函数用于连接“地址计算语句”和“内存访问语句”之间的数据流关系。
8. buildBlockGraph是整个文件中最核心的函数，其主要任务是为单个基本块构建高级数据流图 AdvDFG。函数首先遍历基本块中的所有语句，并忽略 PHI 和 LABEL 等非计算性节点。随后创建一个入口节点，并按照语句顺序为每条指令创建对应的数据流图节点，同时建立顺序执行边，反映程序原有的执行顺序。在节点创建过程中，函数会记录每个临时变量最近一次出现的定义节点。当发现 PTR_CALC 指令时，会为其分配独立的链编号，用于表示地址计算链。之后函数会再次扫描所有节点，对于 LOAD 和 STORE 指令：找到其依赖的地址变量；找到产生该地址变量的 PTR_CALC 节点；建立地址依赖边；记录对应的 Chain 信息。最后函数根据每个节点使用的变量集合，查找这些变量对应的定义节点，并建立 Def-Use 数据依赖边。因此最终生成的数据流图同时包含：
 - 顺序执行边
 - Def-Use 数据依赖边
 - 地址计算链依赖边

相比传统控制流图能够更准确地刻画程序中的数据传递关系。

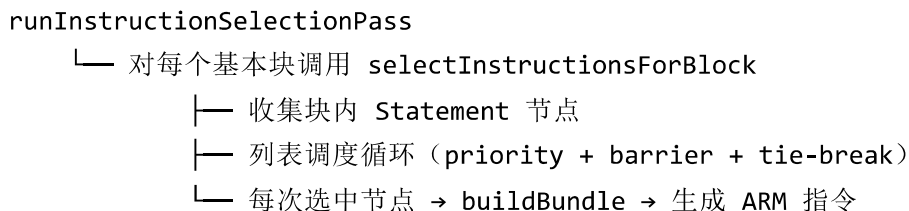
9. buildAdvDFGprog函数是整个 AdvDFG 构建过程的入口函数。函数从 QuadProgram 开始遍历：遍历所有函数；为每个函数创建对应的 advDFGfunc 对象；遍历函数中的所有基本块；调用 buildBlockGraph 为每个基本块构建数据流图；将结果组织成程序级别的 advDFGprog 结构返回。最终形成如下层次结构：



这样整个程序中的每个函数和基本块都拥有对应的数据流图表示，为后续的数据流分析、依赖分析以及优化变换提供基础支撑。

selectInstr.cc

这个文件的整体作用是将前一阶段已经完成指令选择和预调度的基本块序列，进一步线性化为最终的 ARM 汇编指令流，并处理函数布局、Phi 消除、分支跳转、函数调用和栈帧管理。但是块尾的控制流不在此处理，由 `schedule.cc` 负责。它的总体架构如下：



整体设计上，采用**两阶段分离**，emit 负责「翻译对不对」，调度负责「顺序合不合法、是否与参考一致」。并且实现**PTR_CALC 链优化**，尽量 fold 进寻址模式，减少多余 `add / mov`；复杂场景（多 `call`、`mul` 穿插、`NAME store`）才显式 materialize。采用了**启发式调度**，在 DFG 合法前提下，用 `priority + barrier + tie-break` 匹配期望的指令顺序；大量规则针对 `call/malloc/load/store` 的实际组合。保证了**块内局部性**，DFG 和调度均在基本块内完成，块间依赖通过“块外 temp 初始就绪”处理；PHI 与控制流由 `schedule.cc` 负责。

该文件中的核心函数的思路如下：

1. `runInstructionSelectionPass` 是整个 pass 的入口。遍历 `advDFG` 与 `preSchedule` 中每个函数的每个基本块，调用 `selectInstructionsForBlock`；同时维护函数级 scratch temp 计数器 `nextTempNum`，供常量物化、`NAME` 地址等临时使用。
2. `selectInstructionsForBlock` 是块内列表调度主循环，是本文件最核心的控制逻辑。首先用 `statementNodesInOrder` 收集所有 `Statement` 节点（不含块尾控制流）。然后初始化 `definedTemps`：块外已定义的 temp 视为就绪，块内 temp 需等定义后才可用。每轮在所有**依赖已满足**的节点中，选 `effectiveSchedulePriority` 最小者；同 `priority` 时用 `tie-break`（`mul/add` 顺

序、quad 原始顺序等)。用 `moveExtCallBarrierAllows` 过滤非法候选 (如 `extcall` 与常量 `move` 乱序、`load-call` 顺序错误等)。然后调用 `buildBundle` 生成指令, 追加到 `preScheduleBlock`, 并更新 `definedTemps`。若无就绪节点, 强制选第一个未调度节点, 避免死锁。

3. `buildBundle` 函数是指令选择的核心, 按 Quad 类型把一条语句翻译成一个 `StatementBundle`, 即若干 ARM 指令。为了省篇幅, 用下表表示:

Quad 类型	处理方式
LOAD	有 PTR_CALC 链 → <code>emitLoadWithChain</code> 或 <code>emitLoadWithExplicitPtr</code> ; 否则直接 <code>ldr [src]</code>
STORE	有链 → <code>emitStoreWithChain</code> ; 否则 <code>emitStoreDirect</code>
PTR_CALC	仅当 <code>ptrCalcNeedsExplicitBundle</code> 为真时 → <code>emitPtrCalcBundle</code> ; 否则空 bundle (地址 fold 进 load/store)
MOVE	常量用 <code>mov #imm</code> 或 <code>movw</code> ; temp 用 <code>mov</code>
MOVE_BINOP	<code>+</code> / <code>-</code> / <code>*</code> 映射为 <code>add</code> / <code>sub</code> / <code>mul</code> , 尽量 fold 立即数
MOVE_EXTCALL / EXTCALL	<code>emitExtCallArgs</code> + <code>b1</code> + (有返回值时) <code>mov dst, r0</code>
MOVE_CALL / CALL	<code>emitCallArgs</code> + <code>b1</code> / <code>blx</code> (虚调用)

4. `ptrCalcNeedsExplicitBundle` 函数决定 PTR_CALC 是否需要单独生成地址计算指令, 还是 fold 进 `ldr / str [base, #off]`。主要的考虑有:

- `offset ≥ 8` 或大偏移 → 显式 `add`
- 链只被 LOAD 使用 → 通常 fold
- 块内 ≥ 2 个 `call` 且 LOAD 在 `call` 前 → 显式 `materialize` (如 `fibonacci` 递归)
- STORE 用户为 `temp/NAME`、有 `mul` 穿插等 → 显式或推迟

这是寻址优化的关键决策点。

5. `schedulePriority` 与 `effectiveSchedulePriority` 写了两层优先级机制。 `schedulePriority`: 按语句类型给基础优先级 (数字越小越优先):

- 0: 常量 `move`、`call` 参数 `move` (需尽快准备 `r0-r3`)
- 1: 常量加法 `binop`、部分 PTR_CALC
- 2-3: 一般 `move`、`load`、`mul`、`ptr calc`
- 4: 应推迟的 `load/add` (如等第一次 `call` 完成)

`effectiveSchedulePriority`: 在基础优先级上叠加上下文调整, 例如递归块中, 链式 `load` 等第一次 `call` 后再调度; `malloc` 后 `NAME store` 未完成时, 推迟链 `base` 的 `move/ptr calc`; PTR_CALC 与 STORE 之间有未调度 `mul` 时, 推迟 `ptr calc`

6. `moveExtCallBarrierAllows` 函数是在判断调度的合法性, 即使 `priority` 最优, 也可能因语义/寄存器约束被拒绝。典型场景比如常量 `move` 不能抢在紧邻的 `extcall` 前除非 `extcall` 已就绪; `load` 结果 `move` 给 `call` 对象时, 需等 `load-fed call` 条件满足; 链 `base` 的 `move/ptr calc` 需等前面

MOVE_EXTCALL (如 malloc) 及配套 STORE 完成; 双 call 块中, 链式 load 需等第一个 call 调度完。

7. 我还实现了一系列指令生成函数。

emitLoadWithChain / emitStoreWithChain 利用 PTR_CALC 链生成 ldr / str, 优先 fold 为 [base]、[base, #imm]; 变长偏移用 [base, offTemp]; STORE 还需处理 temp/const/NAME 三种源操作数。

emitPtrCalcBundle 显式地址计算: mov (offset=0)、add base, #off 或 add base, offTemp。

emitExtCallArgs / emitCallArgs 把前四个参数 mov 到 r0 - r3, 符合 ARM 调用约定。

emitStoreDirect 无 PTR_CALC 链时直接 str; NAME 源先用 adr 取地址。

8. 此外, 其他的一些辅助函数在这里做简述。

实现 PTR_CALC 链分析: findChainDefNode、ptrCalcConstOffset、ptrCalcBaseTempNum 等从 PTR_CALC 和 advDFG 中提取链 ID、偏移、基址、目的

temp。chainUsedOnlyByLoad、hasTempStoreChainAtOffsetZero、isChainCompanionPtrCalc 等分析链的用户类型, 支撑 fold 与 explicit 的决策。

实现块内模式识别: 识别 malloc (isMallocMoveExtCall、tempIsMallocResultInBlock)、call 参数 (moveUsedAsCallArgumentAfter、stmtUsesTempAsCallArg)、pending NAME store (firstPendingNameStoreIndex、pendingNameStoreToMoveSrc)、load→call 对象传递 (loadFedCallObjectShouldDefer、blockHasUnscheduledLoadFedCall) 等, 供 schedulePriority 和 barrier 使用。

实现 Mul/Add 调度辅

助: hasUnscheduledMulBefore/After、hasInterleavedMulBetween、isMulBinopStmt、isAddBinop 等: 在 mul 与 add 交错时决定谁先调度, 保证循环 IV 等场景下指令顺序正确。以及一些基础工

具: makeTemp、appendOper、materializeConstMovw、fitsMovImmediate、

allocateScratchTemp: 构造操作数、追加指令、物化常量、分配 scratch temp, 被各 emit 函数共用。

schedule.cc

本模块负责将预调度结果转换为最终线性化汇编代码。实现过程中首先为函数生成标准 ARM 栈帧序言与尾声, 并按照调用约定完成参数传递; 随后采用深度优先方式展开控制流图, 对基本块进行布局, 并插入 Phi 消除产生的拷贝指令; 对于条件跳转、函数返回和退出调用等控制流语句, 则生成对应 ARM 分支与调用指令。最终将所有基本块组织为顺序汇编指令流, 完成从控制流图到目标代码的转换。具体函数如下:

1. makeTemp函数用于根据给定的临时变量编号创建新的 Temp 对象。在代码生成过程中, 经常需要为常量加载、比较运算等操作分配额外的临时寄存器, 因此该函数为后续汇编生成提供统一的临时变量创建接口。
2. isExitCallStmt函数用于判断一条 Quad 语句是否对应对 exit 函数的调用。由于中间表示中存在普通调用、外部调用以及带返回值调用等多种形式, 因此函数分别检查不同类型的调用节点, 并统一识别程序终止语句, 为后续控制流处理提供支持。

3. `appendOper`函数负责向当前函数的汇编指令序列中添加一条普通操作指令。函数封装了目标寄存器、源寄存器以及跳转目标等信息，是整个调度阶段生成汇编代码最常用的输出接口。
4. `appendLabel`函数用于向汇编代码中插入标签指令。在线性化基本块时，每个基本块入口都会对应一个标签，从而为后续跳转指令提供目标位置。
5. `appendPrologue`函数负责生成函数序言代码。其主要作用是保存被调用者保存寄存器、建立新的栈帧并初始化帧指针，从而为函数执行提供独立的运行环境。
6. `appendEpilogue`函数负责生成函数尾声代码。其主要任务是恢复栈帧、恢复保存寄存器以及返回调用者，与函数序言形成对应关系，保证函数能够正确结束执行。
7. `appendParamMoves`函数用于处理函数入口参数。根据 ARM 调用约定，前四个参数通过寄存器传递，因此该函数会将参数寄存器中的值移动到编译器内部使用的临时变量中，便于后续统一处理。
8. `blockByLabel`函数根据标签查找对应的基本块。由于控制流跳转通常通过标签表示，因此在基本块布局阶段需要利用该函数完成标签到基本块对象的映射。
9. `scheduleForBlock`函数用于查找指定基本块对应的预调度结果。后续生成汇编时需要访问该基本块已经完成的指令选择结果以及控制流信息，因此需要通过该函数建立映射关系。
10. `scheduleForFunc`函数用于查找函数对应的预调度结构。其作用与 `scheduleForBlock` 类似，只不过查询对象由基本块提升到整个函数层次。
11. `findPhiSourceTemp`函数用于在 Phi 函数中寻找指定前驱块对应的输入变量。由于不同前驱边可能对应不同的变量来源，因此该函数为后续 Phi 消除过程提供正确的数据来源信息。
12. `emitPhiCopiesForEdge`函数负责在控制流边上生成 Phi 消除所需的复制指令。通过将 SSA 形式中的 Phi 函数转换为普通赋值语句，实现从 SSA 表示向普通汇编代码的过渡。
13. `allocateScratchTemp`函数用于分配新的临时变量编号。当代码生成过程中需要额外寄存器保存中间结果时，可以通过该函数获得新的虚拟寄存器编号。
14. `materializeOperand`函数负责将 Quad 操作数转换为可直接参与汇编生成的寄存器形式。对于临时变量直接返回对应寄存器，对于常量则先生成加载指令，再返回存放该常量的新临时寄存器。
15. `branchMnemonicForRelop`函数负责将 Quad 中的关系运算符映射为对应的 ARM 条件跳转指令。例如将 `>` 转换为 `bgt`，将 `==` 转换为 `beq`，从而为条件分支生成提供统一接口。
16. `emitCompareAndBranch`函数负责生成条件跳转对应的汇编代码。其主要过程包括操作数准备、比较指令生成以及条件分支指令生成，从而实现高级条件语句向底层分支指令的转换。
17. `emitReturn`函数用于生成函数返回代码。若返回语句包含返回值，则首先将结果移动到 ARM 约定的返回寄存器 `r0` 中，然后输出函数尾声完成函数返回。
18. `exitCallArgs`函数用于提取 `exit` 调用对应的参数列表。由于 `exit` 在中间表示中可能对应多种调用形式，因此该函数统一封装了参数访问逻辑。
19. `emitExitCall`函数负责生成 `exit` 调用对应的汇编代码。首先将退出码放入参数寄存器，然后生成对 `exit` 函数的调用指令，实现程序终止功能。
20. `emitSelectedInstructions`函数用于输出上一阶段指令选择生成的汇编指令序列。其作用是将已经完成模式匹配的目标指令加入最终线性化结果中。
21. `layoutFunction`函数是整个调度模块的核心函数，负责将控制流图展开为线性汇编代码。函数从入口基本块开始递归遍历控制流图，在输出基本块指令的同时处理函数序言、Phi 消除、条件跳转、无条件跳转、返回语句以及退出调用等控制流结构，最终形成完整的汇编执行顺序。

22. `scheduleProg`函数是整个调度阶段的入口函数。它遍历程序中的所有函数，获取对应的预调度结果，并调用 `layoutFunction` 完成每个函数的线性化布局，最终生成完整的目标汇编程序结构。

开发过程和测试结果

开发过程如下：

master github feat: 补充了一点点exit 处理，与buildAdvDFG作对齐 feat: 重新对函数顺序和部分汇编执行顺序做了微调，以更加符合示例思路 feat: 修正了目前的输出，方便后续改进和对比 feat: 完成HW11的指令转换基本逻辑，并完善了列表调度与寻址规则	31 May 2026 00:41 23300240034-chen... 83ddb89 31 May 2026 00:33 23300240034-chen... 57957fd4 30 May 2026 22:30 23300240034-chen... e37fca0c 30 May 2026 22:18 23300240034-chen... 2f068079
---	---

`make run` 之后得到的所有测试的结果，与老师给出的参考测试结果完全一致。并且在终端得到的输出片段如下：

```

block[2] entry=L103
  phi_count=0
  selected_instruction_count=0
  last_instruction_kind=CJUMP
block[3] entry=L107
  phi_count=0
  selected_instruction_count=8
    instr kind=I_OPER assem="movw `d0, #3" dst=<t115> src=<>
    instr kind=I_OPER assem="mul `d0, `s0, `s1" dst=<t11000> src=<t115,t10201>
    instr kind=I_OPER assem="add `d0, `s0, #2" dst=<t10102> src=<t11000>
    instr kind=I_OPER assem="mov r0, `s0" dst=<> src=<t10102>
    instr kind=I_OPER assem="bl putint" dst=<> src=<>
    instr kind=I_OPER assem="movw `d0, #32" dst=<t116> src=<>
    instr kind=I_OPER assem="mov r0, `s0" dst=<> src=<t116>
    instr kind=I_OPER assem="bl putch" dst=<> src=<>
  last_instruction_kind=JUMP
block[4] entry=L108
  phi_count=0
  selected_instruction_count=1
    instr kind=I_OPER assem="mov `d0, #4" dst=<t10103> src=<>
  last_instruction_kind=JUMP
block[5] entry=L109
  phi_count=1
  selected_instruction_count=1
    instr kind=I_OPER assem="sub `d0, `s0, #1" dst=<t10202> src=<t10201>
  last_instruction_kind=JUMP
block[6] entry=L104
  phi_count=0
  selected_instruction_count=5
    instr kind=I_OPER assem="mov r0, `s0" dst=<> src=<t10101>
    instr kind=I_OPER assem="bl putint" dst=<> src=<>
    instr kind=I_OPER assem="movw `d0, #10" dst=<t117> src=<>
    instr kind=I_OPER assem="mov r0, `s0" dst=<> src=<t117>
    instr kind=I_OPER assem="bl putch" dst=<> src=<>
  last_instruction_kind=RETURN
root@cyx:~/fducompilerh2026/fducompilerh2026/HW11#

```

参考资料

- 虎书中文版，主要参考学习第9章目标代码生成的指令选择与地址计算，第10章的基本块内优化思想，以及第11章的列表调度。SSA的PHI消解采用边复制，与第8章中间表示及控制流处理相关。
- 大模型：DeepSeek/Cursor/ChatGPT，协助理解虎书内容和修改代码中遇到的寻址与 PTR_CALC 链以及指令顺序的问题。